SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA1304**

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Simulation

Code of Conduct:

I B.Nynika (192311075) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
B.Nynika

SIMATS ENGINEERING
ASSESSMENT TOOL

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed, Opening, Open,* and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

SOLUTION:

Step 1: Identify States

Let

- **q_0 = Closed** (Start State)
- **q_1 = Opening**
- **q_2 = Open**
- **q_3 = Closing**

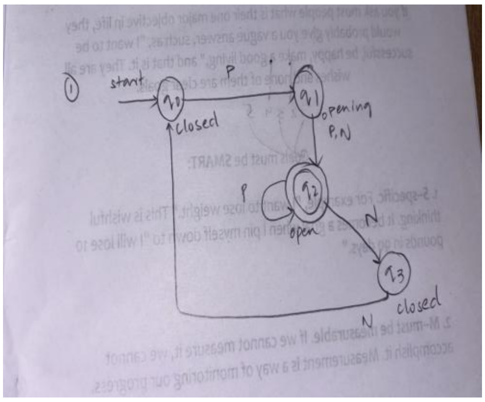
Step 2: Input Symbols

$$\Sigma = \{P, N\}$$

Where

- **P = Person Detected**
- **N = No Person Detected**

Step 3: Draw the NFA



Explanation

- Initially the door is **Closed (q_0)**.
- If a person is detected (**P**), it starts **Opening (q_1)**.
- After opening, it reaches **Open (q_2)**.
- While open:
  - If another person is detected (**P**), it **remains Open**.
  - If no person is detected (**N**), it starts **Closing (q_3)**.
- After closing, it returns to **Closed (q_0)**.

Step 4: Transition Table

Present State	**P**	**N**
→q_0 (Closed)	q_1	—

Present State	P	N
q_1 (Opening)	q_2	q_2
q_2 (Open)	q_2	q_3
q_3 (Closing)	—	q_0

Step 5: Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

- **Q = $\{q_0,q_1,q_2,q_3\}$**
- **$\Sigma = \{P,N\}$**
- **Start State = q_0**
- **Final State = $\{q_2\}$** (Door is Open)

Step 6: Working
Example input:
P N N
q0 --P--> q1
q1 --N--> q2
q2 --N--> q3
q3 --N--> q0
The door opens when a person arrives and closes when no person is detected.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

SOLUTION:
Step 1: Identify States
Let

- **q_0 = Red**
- **q_1 = Green**
- **q_2 = Yellow**

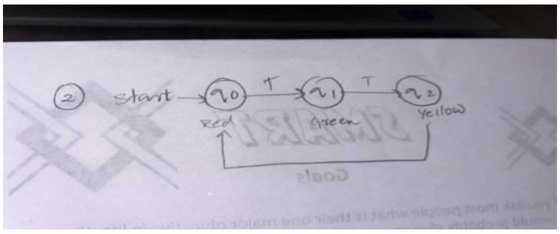
Step 2: Input Alphabet
Use

$$\Sigma = \{T\}$$

where

- **T = Timer expires** (time is over, move to the next light)

Step 3: DFA Diagram



Step 4: Transition Table

Present State	**Input (T)**	**Next State**
→q_0 (Red)	T	q_1 (Green)
q_1 (Green)	T	q_2 (Yellow)
q_2 (Yellow)	T	q_0 (Red)

Step 5: Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

- **Q = {q_0, q_1, q_2}**
- **Σ = {T}**
- **Start State = q_0**
- **Final State = {q_0}** (or any state can be treated as accepting since the system runs continuously)

Step 6: Working

Suppose the timer expires three times.
Start

↓

Red

↓

Green

↓

Yellow

↓

Red
The cycle repeats continuously.

Step 7: Extension
Pedestrian Crossing
When the pedestrian button is pressed (**P**):

- Finish the current Green phase.
- Change to Yellow.
- Then Red.
- Allow pedestrians to cross.
- Resume the normal cycle.

SIMATS ENGINEERING
ASSESSMENT TOOL

Emergency Vehicle

If an emergency vehicle (**E**) is detected:

- Immediately switch the required direction to Green.
- Keep it Green until the emergency passes.
- Resume the normal sequence.

Final Answer (Exam)

States

- q_0 = Red
- q_1 = Green
- q_2 = Yellow

Alphabet

$$\Sigma = \{T\}$$

DFA Transition Table

State	T
$\rightarrow q_0$	q_1
q_1	q_2
q_2	q_0

3 Design DFA using simulator to accept the string the end with ab over set {a,b)

W= aaabab

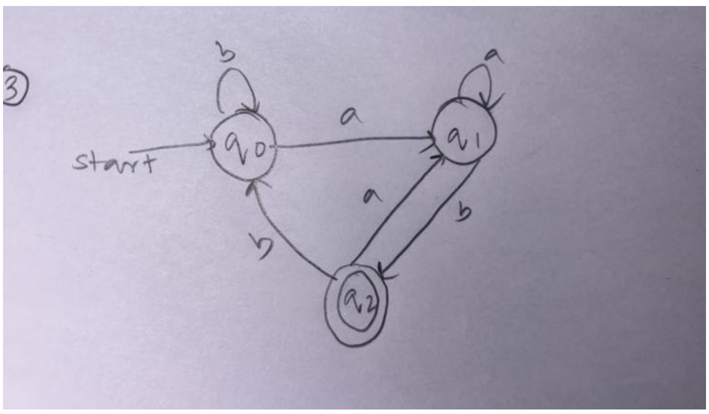
SOLUTION:

Step 1: Identify States

Let

- **q_0** = Start state (no useful suffix matched)
- **q_1** = Last symbol is **a**
- **q_2** = Last two symbols are **ab** (**Final state**)

Step 2: DFA Diagram



Step 3: Transition Table

Present State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Step 4: Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

- **$Q = \{q_0, q_1, q_2\}$**
- **$\Sigma = \{a, b\}$**
- **Start State = q_0**
- **Final State = $\{q_2\}$**

Step 5: Test the String

Given:

$$W = \text{aaabab}$$

Process each symbol:

Symbol	State
Start	q_0
a	q_1
a	q_1
a	q_1
b	q_2

SIMATS ENGINEERING ASSESSMENT TOOL

Symbol	**State**
a	q_1
b	q_2 ✅

Step 6: State Path

q0

|

a

▼

q1

|

a

▼

q1

|

a

▼

q1

|

b

▼

q2

|

a

▼

q1

|

b

▼

q2 ✓ Accepted

Since the final state is **q_2**, the string **aaabab** is **Accepted** because it ends with **"ab"**.

Final Answer (Exam)

DFA Transition Table

State	**a**	**b**
→q_0	q_1	q_0
q_1	q_1	q_2
*q_2	q_1	q_0

Final State

$$F = \{q_2\}$$

Result

$$\boxed{\text{The string } aaabab \text{ is Accepted.}}$$

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

SOLUTION:

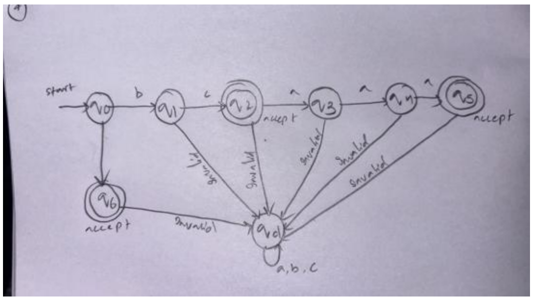
Step 1: Identify States

Let

- **q_0** = Start state
- **q_1** = Read **b**
- **q_2** = Read **bc** (**Accept**)

- **q_3** = Read **bca**
- **q_4** = Read **bcaa**
- **q_5** = Read **bcaaa** (**Accept**)
- **q_6** = Read **c** (**Accept**)
- **qd** = Dead state

Step 2: DFA Diagram



Step 3: Transition Table

Present State	a	b	c
$\rightarrow q_0$	qd	q_1	q_6
q_1	qd	qd	q_2
$*q_2$	q_3	qd	qd
q_3	q_4	qd	qd
q_4	q_5	qd	qd
$*q_5$	qd	qd	qd
$*q_6$	qd	qd	qd
qd	qd	qd	qd

Step 4: Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

States

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, qd\}$$

SIMATS ENGINEERING
ASSESSMENT TOOL

Alphabet

$$\Sigma = \{a, b, c\}$$

Start State

$$q_0$$

Final States

$$F = \{q_2, q_5, q_6\}$$

Step 5: Verification

String = c

q0 → q6 ✓ Accepted

String = bc

q0 → q1 → q2 ✓ Accepted

String = bcaaa

q0 → q1 → q2 → q3 → q4 → q5 ✓ Accepted

String = bcaa

q0 → q1 → q2 → q3 → q4

Ends at q4 (Not Final)

Rejected ❌

String = abc

q0 → qd

SIMATS ENGINEERING ASSESSMENT TOOL

Rejected ❌

Final Answer (Exam)

DFA Transition Table

State	a	b	c
→q_0	qd	q_1	q_6
q_1	qd	qd	q_2
*q_2	q_3	qd	qd
q_3	q_4	qd	qd
q_4	q_5	qd	qd
*q_5	qd	qd	qd
*q_6	qd	qd	qd
qd	qd	qd	qd

Accepting States

$$\boxed{q_2, q_5, q_6}$$

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

SOLUTION:

tep 1: Identify States

Let

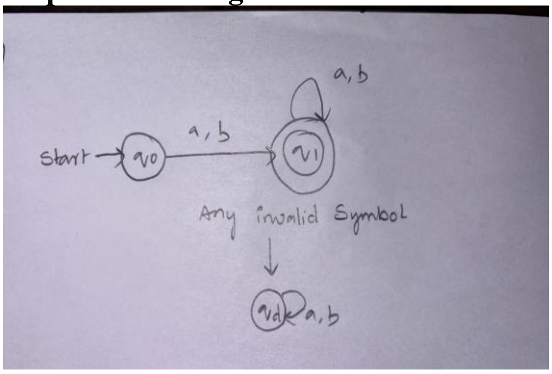
- **q_0** = Start State
- **q_1** = Accepting State (after reading the first symbol)
- **qd** = Dead State (optional, for invalid symbols if required by the simulator)

Step 2: Input Alphabet

$$\Sigma = \{a, b\}$$

SIMATS ENGINEERING ASSESSMENT TOOL

Step 3: DFA Diagram



Step 4: Transition Table

Present State	a	b
$\rightarrow q_0$	q_1	q_1
$*q_1$	q_1	q_1
qd	qd	qd

Step 5: Formal Definition

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

- **$Q = \{q_0, q_1, qd\}$**
- **$\Sigma = \{a,b\}$**
- **Start State = q_0**
- **Final State = $\{q_1\}$**

Step 6: Verification

String = a

q0 → q1 ✓ Accepted

String = b

q0 → q1 ✓ Accepted

String = abba

q0 → q1 → q1 → q1 → q1

Accepted ✓

String = baab

q0 → q1 → q1 → q1 → q1

Accepted ✓

String = ε (Empty String)

q0

Not a final state

SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand